



Programme d'enseignement

Développement Web

Classe passerelle — hepia

Enseignante	Anne Terrier
Durée	20 semaines
Volume horaire	3h théorie + 4h pratique par semaine soit 60h cours + 80h TP = 140h
Public	Étudiant-e-s souhaitant intégrer la 1 ^{re} année hepia, filière ISC
Prérequis	Aucun prérequis en informatique
Stack	HTML5 · CSS · JavaScript · Node.js / Express · MySQL
Fil rouge	Projet MiniForum (forum simplifié)
Version	année académique 2026–2027

Cours donné en parallèle d'un module de POO (C#) et d'un module Linux / Bash

Table des matières

1	Présentation générale	3
2	Alignement avec la 1^{re} année hepia	3
3	Répartition horaire	5
4	Compétences visées	6
5	Le projet fil rouge : MiniForum	7
5.1	Progression en quatre phases	7
5.2	Dépôts de référence et rattrapage	7
6	Modalités d'évaluation	8
6.1	Pondération	8
6.2	Grille d'évaluation du projet final	8
6.3	Grille d'évaluation de la soutenance	8
7	Usage de l'intelligence artificielle générative	9
7.1	Pourquoi	9
7.2	Ce qui reste autorisé	9
7.3	Modalités de contrôle	9
8	Détail semaine par semaine	10
8.1	Phase 1 — HTML, CSS & JavaScript front-end (semaines 1 à 7)	10
8.2	Phase 2 — Bases de données & SQL (semaines 8 à 11)	15
8.3	Phase 3 — Node.js / Express & API REST (semaines 12 à 17)	19
8.4	Phase 4 — Authentification, sécurité & soutenance (semaines 18 à 20)	24
9	Environnement technique	27
10	Ressources	27
10.1	Documentation de référence	27
10.2	Ressources d'appui	27
10.3	Documents fournis par l'enseignante	27

1 Présentation générale

Ce cours constitue le volet **développement web** du programme de la classe passerelle. Il vise l'acquisition des compétences nécessaires à la réalisation d'une application web dynamique complète, de la mise en forme d'une page HTML jusqu'à la persistance des données en base relationnelle.

Le public est composé d'étudiant·e·s souhaitant intégrer la 1^{re} année hepia, filière ISC, dans leur grande majorité sans expérience préalable en informatique. La progression est construite en conséquence : les concepts fondamentaux (variables, conditions, boucles, fonctions) sont introduits progressivement, le retour visuel immédiat dans le navigateur servant de moteur de motivation en début de parcours.

Cours parallèles à prendre en compte

- **Programmation orientée objet (C#)** — les concepts de variable, boucle, fonction et structure de contrôle y sont couverts.
- **Linux / programmation Bash** — les étudiant·e·s sont à l'aise dans un terminal, ce qui rend l'environnement Node.js naturel et cohérent.

Coordination nécessaire avec le module de POO

Ce plan suppose que les **structures de contrôle et les fonctions** aient été traitées en POO (C#) **avant la semaine 6**. Les deux modules étant donnés en parallèle et non en séquence, cette condition doit être confirmée avec le collègue en début d'année. À défaut, la semaine 6 doit être allongée d'une séance au détriment de la semaine 17 (intégration), tenue en réserve à cet effet.

Choix de stack. JavaScript est utilisé à la fois côté client (navigateur) et côté serveur (Node.js / Express), ce qui réduit la charge cognitive liée à l'apprentissage simultané de deux langages. MySQL / MariaDB sert de SGBDR. **Aucun framework front-end n'est introduit** : HTML, CSS et JavaScript standard uniquement. Ce choix est délibéré : l'objectif est la compréhension des mécanismes sous-jacents, pas la maîtrise d'un outil dont la durée de vie est inférieure à celle du cursus.

Travail personnel attendu. Environ 2 à 3 heures par semaine hors présentiel : reprise des notions vues en cours, finalisation des travaux pratiques, préparation des jalons.

2 Alignement avec la 1^{re} année hepia

La classe passerelle prépare l'entrée en filière **Informatique et systèmes de communication (ISC)** de hepia. Il importe de situer honnêtement l'apport de ce cours dans ce cursus.

Le développement web n'est pas enseigné en 1^{re} année hepia. Il apparaît au semestre 4, dans le module ISC_L42 – Développement Web de l'orientation Informatique logicielle. La valeur de ce cours ne réside donc pas dans une avance de programme, mais dans les **fondamentaux transférables** qu'il installe, et dans le fait qu'il donne très tôt une représentation concrète du métier.

Les acquis de ce cours alimentent directement les modules suivants :

Module hepia	Intitulé	Apport de ce cours
ISC_14	Algorithmique et programmation (S1–S2)	Variables, conditions, boucles, fonctions, tableaux et parcours de collections, écrits et débogués des dizaines de fois
ISC_15	Fondements des réseaux et sécurité (S1–S2)	Modèle client/serveur, HTTP, hachage de mots de passe, notion de vulnérabilité applicative
ISC_331	Programmation orientée objet (S3)	Manipulation d'objets et de collections d'objets (conjointement avec le module C#)
ISC_431	Systèmes de bases de données (S4)	Modèle relationnel, clés, jointures, SQL
ISC_L42	Développement Web (S4, orientation logicielle)	Ensemble du cours

Cohérence pédagogique. hepia privilégie l'apprentissage par problèmes et par projet. Le fil rouge MiniForum, mené sur vingt semaines, prépare directement à cette forme de travail : périmètre à tenir, incréments successifs, défense orale du résultat.

Conséquence sur les priorités de ce cours. Puisque la première année hepia porte massivement sur l'algorithmique et la programmation séquentielle, c'est la **partie algorithmique** de ce cours qui est la plus transférable. Elle est traitée en conséquence : deux semaines pleines sur le cœur de JavaScript, et un refus assumé des raccourcis qui masqueraient la logique sous une abstraction.

3 Répartition horaire

Le cours se déroule sur **20 semaines**, à raison de **3 heures de théorie et 4 heures de pratique** par semaine.

Mod.	Thème	Sem.	Cours	TP	Total
A	Fondamentaux du web & environnement	1	3h	4h	7h
B	HTML5 — structure, formulaires, accessibilité	2–3	6h	8h	14h
C	CSS moderne — Flexbox, Grid, responsive	4–5	6h	8h	14h
D	JavaScript — logique, DOM & événements	6–7	6h	8h	14h
E	Bases de données & SQL	8–11	12h	16h	28h
F	JavaScript asynchrone & Express	12–13	6h	8h	14h
G	Node.js / MySQL & CRUD complet	14–15	6h	8h	14h
H	Intégration front / back & robustesse	16–17	6h	8h	14h
I	Authentification & sécurité applicative	18–19	6h	8h	14h
J	Finalisation & soutenance	20	3h	4h	7h
Total		20	60h	80h	140h

Le module E (bases de données) reçoit quatre semaines : c'est la partie la plus abstraite du cours pour un public débutant, et la modélisation ne s'acquiert que par répétition sur des cas variés.

4 Compétences visées

À l'issue de ce cours, les étudiant·e·s seront capables de :

N°	Compétence	Mod.	Évaluation
C1	Structurer une page web sémantique et accessible en HTML5	B	J1, P
C2	Mettre en forme une interface responsive avec CSS (Flexbox, Grid)	C	J1, P
C3	Écrire des scripts JavaScript (ES6+) : variables, fonctions, tableaux, objets	D	J1, E
C4	Manipuler le DOM et réagir aux événements utilisateur	D	J1, P
C5	Concevoir un modèle de données relationnel (UML → MLD)	E	J2, E
C6	Écrire des requêtes SQL avec jointures, agrégations et sous-requêtes	E	J2, E
C7	Écrire du code asynchrone lisible avec <code>async/await</code>	F	J3, E
C8	Construire un serveur Express exposant une API REST	F, G	J3, P
C9	Connecter Node.js à MySQL via des requêtes paramétrées	G	J3, P
C10	Implémenter un CRUD complet de bout en bout	G, H	J3, P
C11	Consommer une API depuis le navigateur avec <code>fetch</code> et gérer les erreurs	H	P
C12	Mettre en œuvre une authentification par session avec hachage	I	P
C13	Prévenir les injections SQL et les failles XSS de base	I	P, S
C14	Présenter oralement un projet technique avec démonstration	J	S

Mod. — lettre du module correspondant dans le tableau de répartition horaire (section 3, A à J).

Évaluation — J1, J2, J3 : jalons intermédiaires · E : contrôle écrit · P : projet final · S : soutenance orale

5 Le projet fil rouge : MiniForum

Plutôt que des exercices isolés par technologie, l'ensemble du cours est structuré autour d'un projet unique qui évolue de semaine en semaine : **MiniForum**, un forum simplifié comportant des utilisateurs, des sujets de discussion (*topics*), des messages (*posts*) et des réactions (*likes*).

Ce périmètre fonctionnel est volontairement restreint, mais suffisamment riche pour illustrer les relations *one-to-many* (un utilisateur a plusieurs messages) et *many-to-many* (un message peut être aimé par plusieurs utilisateurs).

Progression en quatre phases

Phase	Sem.	Thème	Volume	Jalon
1	1–7	HTML / CSS / JS — interface interactive, données en mémoire	49h	J1 (S7)
2	8–11	SQL — modélisation et requêtes sur base réelle	28h	J2 (S11)
3	12–17	Node.js / Express — API REST branchée sur MySQL	42h	J3 (S17)
4	18–20	Authentification, sécurité, soutenance	21h	Soutenance
Total			140h	

La **frustration volontaire** de la phase 1 — les données disparaissent au rafraîchissement — est le principal levier de motivation pour l'introduction de la persistance en base de données.

Dépôts de référence et rattrapage

Un projet filé sur vingt semaines expose à un risque simple : un·e étudiant·e bloqué·e en semaine 4 ne rattrape plus jamais son retard et décroche pour le reste du cours.

Pour l'éviter, un **dépôt de référence** est fourni par l'enseignante à la fin de chaque phase (semaines 7, 11 et 17). Il contient un MiniForum conforme à l'état attendu. Tout·e étudiant·e peut repartir de ce dépôt au début de la phase suivante, sans pénalité sur les jalons ultérieurs.

Ce filet est annoncé aux étudiant·e·s dès la première séance : le but n'est pas de sanctionner un retard, mais d'éviter qu'il ne se transforme en abandon.

6 Modalités d'évaluation

L'évaluation combine un contrôle continu adossé au fil rouge, une épreuve écrite individuelle et une soutenance orale.

Pondération

Épreuve	Objet	Sem.	Poids
Jalon 1	Interface MiniForum interactive (HTML / CSS / JS)	7	10%
Jalon 2	Modèle de données et requêtes SQL	11	10%
Contrôle écrit	HTML, CSS, JavaScript, SQL — individuel, sur machine	12	25%
Jalon 3	API Express connectée à MySQL	17	10%
Projet final	MiniForum complet : code, fonctionnement, qualité	20	30%
Soutenance	Présentation orale et démonstration en direct	20	15%
Total			100%

Les pondérations sont indicatives et doivent être validées au regard du règlement de promotion en vigueur.

Grille d'évaluation du projet final

Critère	Poids
Front-end : HTML sémantique, CSS responsive, accessibilité de base	20%
JavaScript client : manipulation du DOM, <code>fetch</code> , gestion des erreurs	20%
Modèle de données et requêtes SQL	15%
API Express : routes REST, CRUD complet, codes de statut	20%
Sécurité : hachage des mots de passe, requêtes paramétrées, protection XSS	15%
Qualité du code et historique Git (commits réguliers et lisibles)	10%

Grille d'évaluation de la soutenance

Chaque étudiant·e dispose de 10 minutes de présentation suivies de 5 minutes de questions.

Critère	Poids
Démonstration : l'application fonctionne, le parcours est cohérent	30%
Explication du code : l'étudiant·e sait dire ce que fait chaque partie et pourquoi	30%
Réponses aux questions techniques du jury	25%
Clarté de la présentation et gestion du temps	15%

Nature des jalons

Les jalons 1, 2 et 3 sont des points de contrôle courts (une demi-séance) : démonstration de l'état du projet et quelques questions orales sur le code produit. Leur fonction première est **formative** — repérer tôt les difficultés et y remédier — ce qui justifie leur faible poids dans la note finale. Un jalon manqué n'empêche pas de poursuivre le cours, grâce aux dépôts de référence (section 5.2).

7 Usage de l'intelligence artificielle générative

Les assistants de génération de code sont interdits dans ce cours

L'usage de GitHub Copilot, ChatGPT, Claude, Gemini ou de tout autre assistant générant du code est **interdit** pour l'ensemble des travaux remis dans le cadre de ce cours : travaux pratiques, jalons, projet final et contrôle écrit.

Pourquoi

Ce cours a un objectif unique : que des personnes n'ayant jamais programmé sachent programmer à la fin de l'année. Cette compétence s'acquiert en écrivant du code, en le voyant échouer et en comprenant pourquoi. Un assistant qui produit une réponse correcte immédiatement supprime précisément l'étape où l'apprentissage se produit.

L'enjeu est concret : en 1^{re} année hepia, les évaluations d'algorithmique et de programmation sont individuelles et sans assistance. Un·e étudiant·e qui aurait laissé un assistant écrire son MiniForum arriverait avec un projet réussi et aucune des compétences correspondantes.

Ce qui reste autorisé

- La documentation officielle : MDN Web Docs, documentation Express, manuel MySQL
- Les moteurs de recherche et les forums techniques (Stack Overflow), à condition de comprendre et de savoir expliquer tout code repris
- L'entraide entre étudiant·e-s, à condition que chacun·e écrive son propre code
- Les outils d'aide à l'écriture non générateurs de code : correcteur orthographique, formateur automatique (Prettier), analyseur statique (ESLint)

Modalités de contrôle

- Les extensions d'assistance IA sont désinstallées de Visual Studio Code en semaine 1 et leur absence est vérifiée aux jalons
- L'historique Git est examiné à chaque jalon : un projet qui apparaît en un seul commit massif fait l'objet d'un entretien
- Chaque jalon comporte des questions orales sur le code remis
- Le contrôle écrit de la semaine 12 se déroule sur machine, sans accès au réseau

Le manquement à cette règle est traité selon le règlement de l'établissement en matière de fraude.

Note de transparence. Aucun dispositif technique ne détecte l'usage d'un assistant IA de manière fiable. Le contrôle réel de cette règle repose sur les échanges oraux : un·e étudiant·e qui a écrit son code sait l'expliquer, celui ou celle qui l'a fait écrire ne le sait pas. C'est la raison pour laquelle chaque jalon comporte une part orale, et pourquoi la soutenance finale évalue l'explication du code autant que son fonctionnement. Cette interdiction porte sur la période d'apprentissage des fondamentaux, non sur la pratique professionnelle. Les usages en contexte professionnel sont abordés en semaine 20, dans le bilan de fin de cours.

8 Détail semaine par semaine

Phase 1 — HTML, CSS & JavaScript front-end (semaines 1 à 7)

Les modules A à D couvrent les fondamentaux du web côté client. Le fil rouge progresse d'une page HTML statique vers un forum interactif en JavaScript, avant de buter sur la limite de la mémoire volatile. Le cœur de JavaScript reçoit deux semaines pleines : c'est la partie la plus difficile pour un public débutant et la plus utile pour la suite du cursus.

Semaine 1 [A] Fondamentaux du web & environnement

3h cours / 4h TP

Objectifs pédagogiques

- Comprendre le modèle client/serveur et le rôle des technologies web
- Installer et configurer un environnement de développement fonctionnel
- Distinguer HTML, CSS et JavaScript dans leurs rôles respectifs

Cours

- Fonctionnement du web : navigateurs, serveurs, requêtes HTTP/HTTPS
- Vocabulaire fondamental : URL, DNS, protocole, statique et dynamique
- Tour d'horizon de la stack : HTML, CSS, JavaScript, Node.js, SQL
- Installation : VS Code, extensions, Node.js, terminal
- Git : `init`, `add`, `commit`, `push` et **rôle du** `.gitignore`
- Règles du cours : fil rouge, jalons, interdiction des assistants IA

Travaux pratiques

- Configuration complète de l'environnement, désinstallation des extensions d'assistance IA
- Création du dépôt Git MiniForum avec un `.gitignore` correct dès le premier commit

Fil rouge — MiniForum : Mise en place du projet

Création de la structure `miniforum/` (`index.html`, `style.css`, `app.js`) et premier commit Git. Les étudiant·e·s ouvrent leur projet dans le navigateur pour la première fois.

Semaine 2 [B] HTML5 — Structure & sémantique**3h cours / 4h TP****Objectifs pédagogiques**

- Écrire une page HTML5 valide et sémantique
- Structurer le contenu avec les éléments appropriés

Cours

- Structure d'un document : doctype, head, body, métadonnées
- Éléments sémantiques : header, nav, main, section, article, footer
- Texte, liens, images, listes, tableaux
- Hiérarchie des titres et structure du document
- Validation W3C et lecture des messages d'erreur

Travaux pratiques

- Squelette HTML du MiniForum : en-tête, navigation, liste de sujets statique, pied de page
- Page de détail d'un sujet avec une liste de messages codée en dur

Fil rouge — MiniForum : Structure HTML du forum

Les deux pages du MiniForum prennent forme en HTML pur. Aucun style : c'est voulu, la structure doit tenir debout seule.

Semaine 3 [B] HTML5 — Formulaires & accessibilité**3h cours / 4h TP****Objectifs pédagogiques**

- Créer des formulaires complets avec les types d'input HTML5
- Appliquer les principes de base de l'accessibilité web

Cours

- form, input (text, email, password, number, date), textarea, select
- Attributs name, required, placeholder, pattern, maxlength
- Validation native du navigateur et ses limites
- Accessibilité : label, attribut alt, contraste, navigation au clavier
- Aperçu : à quoi servira name lorsque le serveur recevra les données (semaine 15)

Travaux pratiques

- Formulaire de création de sujet et formulaire de réponse
- Formulaires d'inscription et de connexion (structure seule, sans traitement)
- Passage du site au vérificateur d'accessibilité des DevTools

Fil rouge — MiniForum : Tous les formulaires en place

Le MiniForum contient l'intégralité de ses formulaires. Ils ne font encore rien, mais leur structure ne changera plus jusqu'à la fin du cours.

Objectifs pédagogiques

- Appliquer des styles en comprenant la cascade et la spécificité
- Construire des mises en page unidimensionnelles avec Flexbox

Cours

- Sélecteurs, cascade, spécificité, héritage
- Modèle de boîte : `margin`, `padding`, `border`, `box-sizing`
- Flexbox : `display: flex`, axes, alignement, `flex-wrap`, `gap`
- Pseudo-classes : `:hover`, `:focus`, `:nth-child`
- Inspection et modification des styles dans les DevTools

Travaux pratiques

- Mise en forme du MiniForum : typographie, couleurs, espacements
- Barre de navigation et cartes de sujet en Flexbox

Fil rouge — MiniForum : Le forum prend forme visuellement

Le MiniForum devient présentable. C'est généralement la séance où le projet cesse d'être un exercice et devient « leur » site.

Semaine 5 [C] CSS — Responsive, Grid & variables

3h cours / 4h TP

Objectifs pédagogiques

- Adapter une mise en page à différentes tailles d'écran
- Utiliser CSS Grid pour des mises en page bidimensionnelles
- Structurer sa feuille de style avec des variables CSS

Cours

- Media queries et approche *mobile-first*
- CSS Grid : `grid-template-columns`, `grid-template-rows`, `gap`, `grid-area`
- Unités relatives : `rem`, `em`, `%`, `vw` / `vh`
- Variables CSS (`-custom-property`) et thème de couleurs
- Images responsive : `max-width`, `object-fit`

Travaux pratiques

- Extraction de la palette du MiniForum en variables CSS
- Adaptation responsive : mobile, tablette, ordinateur
- Page de détail en Grid : messages à gauche, informations du sujet à droite sur grand écran

Fil rouge — MiniForum : MiniForum responsive

Le forum s'affiche correctement du téléphone à l'écran large. Les variables CSS définies ici sont conservées jusqu'à la soutenance.

Semaine 6 [D] JavaScript — Logique, tableaux & objets

3h cours / 4h TP

Objectifs pédagogiques

- Écrire des scripts JavaScript ES6+ structurés en fonctions
- Manipuler des tableaux d'objets avec les méthodes de parcours
- Débuguer un script avec la console et les points d'arrêt

Cours

- Variables (`let`, `const`), types, opérateurs, conditions, boucles
- Fonctions classiques et fléchées, paramètres, valeur de retour, portée
- Objets littéraux : propriétés, accès, imbrication
- Tableaux d'objets : `map`, `filter`, `find`, `sort`, `forEach`
- Débogage : `console.log`, points d'arrêt, inspection des variables

Travaux pratiques

- Exercices d'algorithmique sur des tableaux de données du forum
- Construction du jeu de données `topics` et `posts` en JavaScript
- Fonctions de tri et de filtrage des sujets, testées dans la console

Fil rouge — MiniForum : Les données du forum en mémoire

Les sujets et messages existent sous forme de tableaux d'objets, avec les fonctions qui les trient et les filtrent. Rien n'est encore affiché : la logique est isolée de l'interface, et c'est un choix de conception qu'il faut savoir justifier.

Semaine 7 [D] JavaScript — DOM, événements & validation**3h cours / 4h TP****Objectifs pédagogiques**

- Manipuler dynamiquement le DOM depuis JavaScript
- Réagir aux événements utilisateur
- Valider un formulaire côté client et afficher des messages d'erreur

Cours

- DOM : `querySelector`, `querySelectorAll`, navigation dans l'arbre
- Création et insertion d'éléments : `createElement`, `append`, `remove`
- `textContent` **plutôt que** `innerHTML` pour tout contenu saisi par un utilisateur — la raison complète sera vue en semaine 19, la règle s'applique dès maintenant
- Événements : `addEventListener`, objet `event`, `preventDefault`
- Validation de formulaire en JavaScript et affichage des erreurs

Travaux pratiques

- Affichage dynamique de la liste des sujets depuis le tableau JavaScript
- Ajout d'un sujet : lecture du formulaire, ajout en mémoire, mise à jour de l'affichage
- Validation des champs avec messages d'erreur sous chaque champ

Fil rouge — MiniForum : Forum interactif...mais volatile

Les sujets s'ajoutent et s'affichent. Constat collectif en fin de séance : tout disparaît au rafraîchissement. Cette frustration déclenche la phase suivante.

Jalon 1 — fin de semaine 7 (10%)

Attendu : un MiniForum en HTML/CSS/JS fonctionnant entièrement dans le navigateur — structure sémantique, mise en forme responsive, affichage dynamique de la liste des sujets depuis un tableau JavaScript, ajout d'un sujet avec validation.

Modalité : démonstration de 10 minutes et questions orales sur le code. Un dépôt de référence est publié à l'issue du jalon.

Phase 2 — Bases de données & SQL (semaines 8 à 11)

Avant d'introduire le serveur, les étudiant·e·s acquièrent les bases du modèle relationnel et de SQL sur une base pré-remplie fournie par l'enseignante. L'objectif est de maîtriser les requêtes avant d'avoir à les écrire dans du code. C'est la partie la plus abstraite du cours, d'où les quatre semaines.

Semaine 8 [E] Modélisation relationnelle (UML → MLD)

3h cours / 4h TP

Objectifs pédagogiques

- Identifier entités, attributs et relations d'un système d'information
- Construire un diagramme de classes et le transformer en modèle logique
- Distinguer relations *one-to-many* et *many-to-many*

Cours

- SGBDR, table, ligne, colonne, clé primaire, clé étrangère
- Notation UML : multiplicités 1, 0..1, 1..*, 0..*
- *One-to-many* : un utilisateur a plusieurs messages
- *Many-to-many* : un message peut être aimé par plusieurs utilisateurs
- Du diagramme au MLD : règles de transformation, tables de liaison
- Types de données MySQL et contraintes d'intégrité

Travaux pratiques

- Modélisation de deux ou trois cas courts (bibliothèque, réservation de salles)
- Construction collective du diagramme de classes du MiniForum
- Transformation en MLD et création des tables en SQL

Fil rouge — MiniForum : Schéma officiel du forum

Le schéma `users / topics / posts / likes` est établi collectivement et sert de référence jusqu'à la fin du cours. Les étudiant·e·s créent eux-mêmes les tables.

Point d'organisation — installation de MySQL

L'installation de MySQL sur les postes de la classe consomme facilement une séance entière de TP. Trois options, à trancher avant la rentrée : une base hébergée sur un serveur de l'école (le plus simple pour les étudiant·e·s), un conteneur Docker fourni clé en main (cohérent avec le module Linux, et utile pour la suite), ou une installation locale guidée réalisée en travail personnel avant la semaine 8. La troisième option est la moins fiable.

Objectifs pédagogiques

- Écrire des requêtes de lecture avec filtres et tris
- Utiliser les fonctions d'agrégation et le regroupement

Cours

- SELECT, WHERE, opérateurs de comparaison et logiques
- ORDER BY, LIMIT, DISTINCT
- LIKE, IN, BETWEEN, gestion de NULL
- Agrégation : COUNT, SUM, AVG, MIN, MAX
- GROUP BY et HAVING, distinction avec WHERE

Travaux pratiques

- Requêtes progressives sur la base MiniForum pré-remplie
- Sujets les plus récents, nombre de messages par sujet, activité par utilisateur

Fil rouge — MiniForum : Interroger les vraies données

La base fournie contient 10 utilisateurs, 20 sujets et 80 messages. Les étudiant·e·s écrivent des requêtes qui produisent exactement ce qu'affiche leur page d'accueil.

Semaine 10 [E] SQL — Jointures**3h cours / 4h TP****Objectifs pédagogiques**

- Croiser plusieurs tables avec `INNER JOIN` et `LEFT JOIN`
- Interroger une relation *many-to-many* via sa table de liaison

Cours

- Principe de la jointure : reconstituer l'information éclatée entre les tables
- `INNER JOIN` et alias de table
- `LEFT JOIN` : les cas sans correspondance, et pourquoi ils comptent
- Jointures multiples et jointure sur une table de liaison
- Combinaison jointure et agrégation
- Lisibilité : indentation, alias explicites, ordre des clauses

Travaux pratiques

- Messages d'un sujet avec le nom de leur auteur
- Sujets sans aucune réponse (cas typique du `LEFT JOIN`)
- Nombre de *likes* par message via la table de liaison

Fil rouge — MiniForum : Toutes les vues du forum en SQL

Chaque écran du MiniForum a désormais sa requête. Ce sont exactement celles qui seront placées dans le serveur Express en semaine 14.

Semaine 11 [E] SQL — Écriture, sous-requêtes & injections**3h cours / 4h TP****Objectifs pédagogiques**

- Écrire des requêtes de modification des données
- Utiliser des sous-requêtes
- Comprendre le principe d'une injection SQL

Cours

- `INSERT INTO`, `UPDATE ...SET ...WHERE`, `DELETE FROM ...WHERE`
- Le `WHERE` oublié : démonstration et conséquences
- Sous-requêtes dans le `WHERE` et dans le `FROM`
- Suppression et intégrité référentielle : `ON DELETE CASCADE`
- Injections SQL : démonstration d'une requête concaténée, et principe de la requête paramétrée

Travaux pratiques

- Exercices CRUD complets sur la base MiniForum
- Requêtes d'analyse : contributeurs les plus actifs, sujets sans réponse
- Exploitation d'une faille d'injection sur une page de démonstration fournie

Fil rouge — MiniForum : Le forum a une vraie mémoire

Les données du MiniForum se créent, se modifient et se suppriment en SQL. Il manque encore l'intermédiaire entre le navigateur et la base : c'est l'objet de la phase 3.

Jalon 2 — fin de semaine 11 (10%)

Attendu : le modèle de données du MiniForum (diagramme de classes et MLD), le script de création des tables, et un jeu d'une dizaine de requêtes couvrant filtres, agrégations, jointures et écriture.

Modalité : remise du script SQL et entretien de 10 minutes sur les choix de modélisation.

Phase 3 — Node.js / Express & API REST (semaines 12 à 17)

C'est la phase la plus dense techniquement. Les étudiant-e-s construisent le serveur qui manquait depuis la phase 1 et y branchent progressivement leur front-end. Le JavaScript des semaines 6 et 7 est immédiatement réinvesti côté serveur. La semaine 17 est délibérément tenue en réserve : elle sert de rattrapage si la classe a pris du retard.

Semaine 12 [F] JavaScript asynchrone & Node.js

3h cours / 4h TP

Objectifs pédagogiques

- Comprendre le modèle d'exécution asynchrone de JavaScript
- Écrire du code asynchrone lisible avec `async / await`
- Exécuter JavaScript hors du navigateur avec Node.js

Cours

- Réactivation express du JavaScript des semaines 6–7 (trois semaines de SQL se sont écoulées)
- Synchrones et asynchrones : pourquoi JavaScript a besoin des *Promises*
- *Promises* : états, `then / catch`
- `async / await` et équivalence avec les *Promises*
- Gestion des erreurs : `try / catch` en contexte asynchrone
- Node.js : `npm init`, `npm install`, scripts du `package.json`

Travaux pratiques

- **Contrôle écrit (2h)** : HTML, CSS, JavaScript et SQL, sur machine, sans réseau
- Premier script Node.js et fonction asynchrone simulant un délai serveur

Fil rouge — MiniForum : Simuler l'appel serveur

Une fonction asynchrone fictive retourne les données du forum après un délai. Le front-end est adapté pour l'attendre avec `await` — syntaxe identique à celle qu'il utilisera avec `fetch` en semaine 16.

Objectifs pédagogiques

- Mettre en place un serveur HTTP avec Express
- Définir des routes GET renvoyant du JSON
- Comprendre les principes d'une API REST

Cours

- Express : structure d'une application, `app.listen`, cycle d'une requête
- Routes et paramètres d'URL (`:id`), objets `req` et `res`
- Principes REST : ressources, verbes HTTP, codes de statut (200, 201, 400, 404, 500)
- *Middleware* : `express.json()`, `express.static()`
- Organisation du code : séparation des routes et de la logique métier
- Lecture d'une documentation de version : différences Express 4 et 5

Travaux pratiques

- Serveur Express servant le front-end statique et exposant GET `/api/topics`
- Routes GET `/api/topics` et GET `/api/topics/:id` avec données codées en dur
- Test des routes avec l'extension REST Client de VS Code

Fil rouge — MiniForum : Le serveur du MiniForum démarre

Le front-end est servi par Express et récupère ses données via `fetch('/api/topics')` au lieu du tableau en dur. Les données restent codées en dur côté serveur : la base arrive la semaine suivante.

Express 5 et les tutoriels en ligne

`npm install express` installe désormais Express 5, alors que la quasi-totalité des tutoriels que les étudiant·e·s consulteront sont écrits pour Express 4. La différence la plus piègeuse : `req.body` vaut `undefined` au lieu de `{}` lorsque `express.json()` n'est pas déclaré, ce qui produit une erreur qu'un débutant ne sait pas interpréter. La version est donc **figée dans le package.json fourni**, et le point est traité explicitement en cours.

Semaine 14 [G] Node.js — Connexion MySQL & requêtes paramétrées**3h cours / 4h TP****Objectifs pédagogiques**

- Connecter un serveur Node.js à MySQL avec `mysql2`
- Écrire des requêtes paramétrées pour prévenir les injections
- Exposer des résultats SQL dans des réponses JSON

Cours

- Pilote `mysql2` : pool de connexions et API *promise*
- Requêtes paramétrées avec `?` : comparaison directe avec la concaténation
- Transformation des résultats SQL en objets JSON cohérents
- Variables d'environnement : `.env`, `dotenv` — et rappel du `.gitignore`
- Gestion des erreurs de connexion et de requête côté serveur

Travaux pratiques

- Connexion du serveur Express à la base MiniForum
- Réutilisation des requêtes écrites en semaine 10 dans les routes GET
- Vérification que `.env` n'est pas suivi par Git

Fil rouge — MiniForum : Les vraies données arrivent dans le navigateur

Le front-end affiche les données réelles de MySQL. Une modification faite directement en SQL apparaît dans l'interface après rafraîchissement : le circuit complet devient visible pour la première fois.

Semaine 15 [G] Express — CRUD complet (POST, PUT, DELETE)**3h cours / 4h TP****Objectifs pédagogiques**

- Implémenter les routes d'écriture dans Express
- Valider les données entrantes côté serveur
- Renvoyer les codes de statut HTTP appropriés

Cours

- Corps de requête : `req.body` et `express.json()`
- POST (création), PUT / PATCH (mise à jour), DELETE (suppression)
- Validation côté serveur : champs requis, types, longueurs — et pourquoi la validation client ne suffit jamais
- Codes de statut et format d'erreur cohérent
- Récupération de l'identifiant créé (`insertId`) et réponse 201

Travaux pratiques

- Routes POST `/api/topics` et POST `/api/topics/:id/posts`
- Routes PUT `/api/posts/:id` et DELETE `/api/posts/:id` avec gestion du 404
- Test complet de l'API au REST Client avant tout branchement du front

Fil rouge — MiniForum : Une API complète

Le MiniForum dispose d'une API REST couvrant l'ensemble des opérations. Elle est testée hors navigateur : le front-end ne sait pas encore s'en servir.

Semaine 16 [H] Fetch côté client & gestion des erreurs

3h cours / 4h TP

Objectifs pédagogiques

- Effectuer des requêtes `fetch` POST, PUT et DELETE depuis le navigateur
- Gérer les erreurs réseau et les réponses d'erreur du serveur
- Synchroniser l'interface avec l'état réel de la base

Cours

- `fetch` avec options : méthode, en-têtes, corps JSON
- Distinction entre erreur réseau et erreur HTTP (`response.ok`)
- Retours utilisateur : état de chargement, succès, échec
- Mise à jour optimiste et mise à jour après confirmation du serveur
- Onglet Réseau des DevTools : lire une requête et sa réponse

Travaux pratiques

- Branchement de tous les formulaires du MiniForum sur l'API
- Affichage de messages d'erreur cohérents en cas d'échec serveur
- Test du comportement de l'interface serveur arrêté

Fil rouge — MiniForum : Le forum est fonctionnel de bout en bout

Créer un sujet, y répondre, supprimer un message : tout persiste. Le circuit HTML → JS → `fetch` → Express → MySQL → JSON → DOM est opérationnel.

Semaine 17 [H] Intégration, refactoring & débogage

3h cours / 4h TP

Objectifs pédagogiques

- Diagnostiquer méthodiquement une panne dans une application complète
- Réorganiser un code devenu difficile à lire

Cours

- Méthode de diagnostic : isoler la couche fautive (navigateur, réseau, serveur, base)
- Lecture d'une trace d'erreur Node.js
- Refactoring : extraction de fonctions, nommage, suppression des répétitions
- Structuration d'un projet Express : dossiers routes, db, public
- Revue de code croisée : lire le projet d'un·e camarade et formuler un retour

Travaux pratiques

- Réorganisation du projet MiniForum selon la structure vue en cours
- Revue de code en binôme, avec restitution
- *Séance de réserve* : si la classe a pris du retard, cette semaine sert au rattrapage

Fil rouge — MiniForum : Un projet propre et lisible

Le MiniForum est réorganisé, les fonctions sont nommées, les fichiers sont séparés. C'est l'état sur lequel se greffera l'authentification.

Jalon 3 — fin de semaine 17 (10%)

Attendu : un MiniForum complet sans authentification — API Express connectée à MySQL, CRUD fonctionnel de bout en bout, requêtes paramétrées, gestion des erreurs côté client, code organisé en fichiers.

Modalité : démonstration et questions sur le code. Dernier dépôt de référence publié à l'issue du jalon.

Phase 4 — Authentification, sécurité & soutenance (semaines 18 à 20)

Phase courte et intense : les concepts de sécurité sont introduits et immédiatement appliqués au MiniForum. La dernière semaine est consacrée à la finalisation et aux soutenances.

Semaine 18 Authentification — Inscription, connexion & sessions

3h cours / 4h TP

Objectifs pédagogiques

- Implémenter une inscription avec hachage de mot de passe
- Mettre en œuvre une authentification par session
- Protéger des routes aux seuls utilisateurs connectés

Cours

- Stockage des mots de passe en clair : cas réels et conséquences
- Hachage et salage : principe, et pourquoi un hachage n'est pas un chiffrement
- Mise en œuvre avec `bcryptjs`
- Sessions côté serveur : `express-session`, cookies, durée de vie
- *Middleware* d'authentification et protection de routes
- Aperçu des alternatives (jetons JWT) sans mise en œuvre

Travaux pratiques

- Routes `POST /api/register` et `POST /api/login`
- Route `POST /api/logout` et *middleware* de protection
- Restriction des routes d'écriture aux utilisateurs connectés
- Affichage conditionnel côté client selon l'état de connexion

Fil rouge — MiniForum : Seuls les membres peuvent poster

Le MiniForum dispose d'une inscription et d'une connexion réelles. Chaque message affiche l'auteur issu de la session. Les mots de passe ne sont jamais lisibles en base.

Pourquoi `bcryptjs` et non `bcrypt`. Le paquet `bcrypt` est un module natif dont la compilation échoue régulièrement selon la machine et la version de Node. `bcryptjs`, écrit en JavaScript pur, s'installe partout sans outil de compilation. L'écart de performance est sans importance à cette échelle, et la séance ne se transforme pas en dépannage collectif.

Objectifs pédagogiques

- Identifier et corriger les vulnérabilités courantes d'une application web
- Finaliser et fiabiliser le projet

Cours

- XSS : démonstration d'une injection de script dans un message de forum
- Prévention : `textContent` plutôt que `innerHTML`, échappement — retour sur la règle posée en semaine 7
- Injections SQL : audit du projet, vérification que toutes les requêtes sont paramétrées
- Contrôle d'autorisation : distinguer authentification et permission (supprimer **son** message)
- Ce qu'on ne stocke pas, ce qu'on ne renvoie pas dans une réponse JSON
- Panorama des vulnérabilités web courantes et ressources pour aller plus loin

Travaux pratiques

- Audit de sécurité croisé : chercher une faille dans le projet d'un·e camarade
- Correction des vulnérabilités trouvées dans son propre projet
- Finalisation : correction des bugs restants, rédaction du README

Fil rouge — MiniForum : Un forum qui tient debout

Le MiniForum résiste aux injections de script et de SQL, et n'autorise chaque utilisateur qu'à modifier ses propres contenus. Le projet est prêt pour la soutenance.

Objectifs pédagogiques

- Présenter oralement un projet technique avec démonstration
- Situer les acquis du cours dans la suite du parcours

Cours

- Méthodologie de soutenance : structure, démonstration en direct, anticipation des questions
- Retour collectif sur les difficultés rencontrées durant les vingt semaines
- Bilan de la stack apprise et perspectives : frameworks, tests, déploiement
- Lien avec la 1^{re} année hepia : ce qui sera réutilisé et sous quelle forme
- Usage professionnel des assistants IA : ce qu'ils permettent une fois les fondamentaux acquis, et pourquoi ils étaient interdits jusqu'ici

Travaux pratiques

- Derniers ajustements et préparation de la démonstration
- Soutenances individuelles : 10 minutes de présentation, 5 minutes de questions

Fil rouge — MiniForum : Soutenance du MiniForum complet

Chaque étudiant·e présente et fait fonctionner son MiniForum : interface responsive, JavaScript client, API Express, base MySQL, authentification et sécurité. Les questions portent sur les choix de conception et les difficultés rencontrées.

9 Environnement technique

Élément	Choix retenu
Éditeur	Visual Studio Code, extensions ESLint, Prettier, REST Client, SQLTools
Navigateur	Firefox ou Chrome, avec les outils de développement
Runtime serveur	Node.js 24 LTS
Gestionnaire de paquets	npm
Framework serveur	Express 5 (version figée dans le package .json fourni)
Base de données	MySQL 8 ou MariaDB
Client SQL	SQLTools dans VS Code, ou DBeaver
Gestion de version	Git + GitHub
Paquets npm	express, mysql2, bcryptjs, express-session, dotenv

Notes de version. Node.js 24 est la version LTS active en 2026 ; Node 20 est en fin de vie et ne doit plus être utilisée. Express 5 est désormais installé par défaut par npm, ce qui crée un décalage avec la documentation en ligne : la version est donc figée et le point est traité en cours (semaine 13).

Git comme fil transversal. Le dépôt Git n'est pas un exercice de la semaine 1 : il est alimenté à chaque séance et son historique fait partie de l'évaluation du projet final. Un projet livré en un commit unique n'est pas conforme.

10 Ressources

Documentation de référence

- **MDN Web Docs** — developer.mozilla.org — référence pour HTML, CSS et JavaScript ; c'est la source à consulter en premier, et son usage est enseigné dès la semaine 1
- **Documentation Express** — expressjs.com — en veillant à consulter la section correspondant à la version 5
- **Manuel MySQL** — dev.mysql.com/doc — pour la syntaxe SQL et les types de données
- **Documentation Node.js** — nodejs.org/docs — pour l'API standard

Ressources d'appui

- **Flexbox Froggy** et **Grid Garden** — jeux courts pour ancrer Flexbox et Grid (semaines 4 et 5)
- **SQLZoo** — exercices SQL progressifs et autocorrigés (semaines 9 à 11)
- **Can I use** — caniuse.com — compatibilité des fonctionnalités selon les navigateurs
- **OWASP Top Ten** — panorama des vulnérabilités web (semaine 19)

Documents fournis par l'enseignante

- Un polycopié par semaine, avec exercices et corrigés
- La base MiniForum pré-remplie (script SQL de création et de peuplement)
- Un dépôt de référence à l'issue de chaque phase (semaines 7, 11 et 17)
- Le package .json du projet, avec les versions figées

Document rédigé par Anne Terrier — cfpt·i Informatique, Genève
Cours Développement Web — Classe passerelle hepia
année académique 2026–2027